

Real-World Agricultural Price Data for Analysis & Forecasting

Domain

Agricultural Market

Objective

1. To analyse mandi price trends of major crops across India from 2023–2025
2. To study price variations across states, districts, and markets
3. To identify seasonal fluctuations in crop prices
4. To compare commodity prices across different regions of India
5. To provide actionable insights for farmers, traders, and policymakers
6. To support decision-making in agricultural marketing

Outcome

The project resulted in identifying price trends, seasonal patterns, and regional variations in agricultural mandi prices. It also produced a cleaned dataset, meaningful visualizations, and a predictive model for forecasting future prices, enabling data-driven decision-making.

Dataset Information

Source: Kaggle

Year / Timeline: Data collected during 2023 to 2025

Dataset Description:

This dataset contains real-world agricultural mandi (market) price data from India collected between 2023 and 2025. The data is sourced from the official Government of India platform, Agmark net (<https://www.agmarknet.gov.in/>), and represents daily commodity prices across multiple states, districts, and markets.

It includes pricing information for various agricultural commodities such as wheat, tomato, onion, potato, and more, along with their varieties and grades.

Dataset Size: Total Rows: 811131 & Total Columns: 10

Features

The dataset includes the following key attributes

State: State name of the Market

District Name: District Name of the Market

Market Name: Name of the Market

Commodity: Commodity like rice, wheat, onion

Variety: Crop type like Rice, varieties can be Basmati, Ponni etc.

Grade: Quality Level of Crop

Min Price: Lowest Market rate

Max Price: Highest Market rate

Modal Price: Most Common Selling Price (Actual market price vary day to day)

Price Date: When the price occurred

Type of Analysis

Descriptive Analysis (summarizing the dataset)

understanding the overall behaviour of agricultural mandi prices before applying advanced techniques

Diagnostic Analysis (finding reasons price change)

To identify the causes of price fluctuations and what are the key factors such as seasonality, regional differences are influencing to increase the mandi prices.

Predictive Analysis (Predict future prices of commodities)

It will be very useful for farmers and traders make better selling and buying decisions

Prescriptive Analysis (recommendations for business decisions)

what action should be taken for farmers, traders, getting maximum profit and min risk

Problem Definition:

Agricultural markets in India face inefficiencies due to lack of data-driven decision-making in the following reasons

Agricultural commodity prices fluctuate frequently due to supply, demand and external factors.

Farmers often sell crops at the wrong time or in the wrong market, leading to lower profits

Lack of data-driven systems results in inefficient price discovery across markets.

There is no reliable mechanism to predict future prices based on historical data

Due to unpredictable prices, farmers face inconsistent and unstable income.

Expected Outcomes

The expected outcomes include identifying price trends, seasonal patterns, regional variations, along with generating accurate predictions and actionable insights to support better decision-making in agricultural markets.

Identify how commodity prices change over time.

Compare prices across different mandis to find the most profitable markets.

Help farmers choose the best time and market to sell their crops.

Understand which commodities have stable vs highly fluctuating prices.

Provide actionable suggestions.

Create interactive charts/Power BI dashboards for easy understanding.

To begin the analysis, I first imported the essential Python libraries required for working with the dataset. These libraries help in performing various tasks such as data cleaning, preprocessing, analysis, and visualization.

The main libraries used in this project are:

Pandas- for data manipulation and analysis

NumPy - for numerical operations

Matplotlib - for creating charts and visualizations

Seaborn for advanced statistical visualizations

Import libraries and Dataset

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
# df = pd.read_csv("Agriculture_price.csv") #
df = pd.read_csv("https://drive.google.com/uc?export=download&id=1p1g5vRvY3QxYOVkBTZwnPPmVzmkOwko")
df
```

	STATE	District Name	Market Name	Commodity	Variety	Grade	Min_Price	Max_Price	Modal_Price	Price Date
0	Maharashtra	nashik	Lasalgaon(Niphad)	Wheat	Maharashtra 2189	FAQ	NaN	NaN	NaN	06-06-2023
1	Maharashtra	NaN	Patan	Tomato	Other	FAQ	1000.0	1500.0	NaN	06-06-2023
2	uttar pradesh	mainpuri	NaN	Potato	NaN	NaN	800.0	820.0	NaN	06-06-2023
3	Rajasthan	chittorgarh	NaN	Wheat	Other	FAQ	2040.0	2668.0	NaN	NaN
4	Rajasthan	NaN	Pratapgarh	NaN	Other	FAQ	476.0	NaN	617.0	06-06-2023
...	...	...	...	...	...	...	...	...	...	...
811126	Tamilnadu	nagapattinam	Nagapattinam(Uzhavar Sandhai)	Onion	NaN	Local	4500.0	5000.0	5000.0	11-01-2024
811127	Punjab	NaN	Doraha	NaN	Other	FAQ	1800.0	2000.0	1900.0	9/19/2024
811128	Haryana	panchkula	Barwala	NaN	Other	NaN	NaN	1000.0	1000.0	NaN
811129	Tamilnadu	NaN	Kariyapatti(Uzhavar Sandhai)	NaN	(Red Nanital)	Local	NaN	NaN	3000.0	4/25/2025
811130	Jammu & Kashmir	NaN	NaN	NaN	Nasik	FAQ	1400.0	1600.0	1500.0	06-09-2023

811131 rows x 10 columns

The Data Frame contains agricultural price data with columns such as State, District Name, Market Name, Commodity, Variety, Grade, Min Price, Max Price, Modal Price, and Price Date.

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 811131 entries, 0 to 811130
Data columns (total 10 columns):
#   Column          Non-Null Count  Dtype
---  -
0   STATE            646326 non-null  object
1   District Name    607858 non-null  object
2   Market Name      607800 non-null  object
3   Commodity        608154 non-null  object
4   Variety          608909 non-null  object
5   Grade            608623 non-null  object
6   Min_Price        608484 non-null  float64
7   Max_Price        607939 non-null  float64
8   Modal_Price      608525 non-null  float64
9   Price Date       608057 non-null  object
dtypes: float64(3), object(7)
memory usage: 61.9+ MB
```

df.info () output gives a quick summary of the dataset structure in Pandas.

Data Cleaning & Pre-processing:

Handle Missing Values:

The df.isnull().sum() function is used to check for missing or null values in Here, is null() identifies whether any value is missing, and sum () counts the total missing values in each column.

The output shows the count of missing values for every column in the dataset

```
df.isnull().sum() #finding the missing value dataframe #
```

```
0
STATE      164805
District Name  203273
Market Name  203331
Commodity   202977
Variety     202222
Grade       202508
Min_Price   202647
Max_Price   203192
Modal_Price  202606
Price Date  203074

dtype: int64
```

Fill Numeric missing values with median if dataset has null values:

The fillna() function replaces missing values in each column using median value of that respective column. In this project so many outliers are occupying the respective numerical columns. This technique helps maintain data consistency and prevents loss of important records during data preprocessing. It is commonly used in Exploratory Data Analysis (EDA) and machine learning to handle null values in numerical data.

```
# fill numerical column Min_Price, Max_Price, Modal_Price columns has outliers .so we can use median #
df['Min_Price'] = df['Min_Price'].fillna(df['Min_Price'].median())
df['Max_Price'] = df['Max_Price'].fillna(df['Max_Price'].median())
df['Modal_Price'] = df['Modal_Price'].fillna(df['Modal_Price'].median())
```

Fill Categorical missing values with mode if dataset has null values:

The code is used to handle missing values in categorical columns of the dataset. The fillna() function replaces missing values with the mode (most frequently occurring value) of that respective column using df[col].mode()[0]. This method helps preserve the most common category in the dataset and maintains data consistency.

```
# fill Categorical Columns #
df['State'] = df['State'].fillna(df['State'].mode()[0]) # return the more than one mode choose the highestst mode #
df['District_Name'] = df['District_Name'].fillna('Unknown')
df['Market_Name'] = df['Market_Name'].fillna('Unknown')
df['Commodity'] = df['Commodity'].fillna('Unknown')
df['Variety'] = df['Variety'].fillna('Unknown')
df['Grade'] = df['Grade'].fillna('Unknown')
```

Handle Duplicates - Check duplicate rows:

The code is used to check whether the dataset contains duplicate rows. The `df.duplicated()` function identifies rows that are repeated in the dataset, and the `sum()` function counts the total number of duplicate records. The output `np.int64(0)` indicates that there are no duplicate rows present in the dataset. This means every record is unique and no repeated employee data exists.

```
# Handling Duplicates : Same State + Same District_Name + Same Market_Name + Same Commodity + Same Price_Date should be unique #
df.duplicated().sum()
```

```
np.int64(284)
```

Remove Duplicates if dataset has duplicates:

The code is used to remove duplicate rows from the dataset if any duplicates are present. The `drop_duplicates()` function identifies repeated records and removes them from the Data Frame. The cleaned dataset is reassigned back to the variable `df`

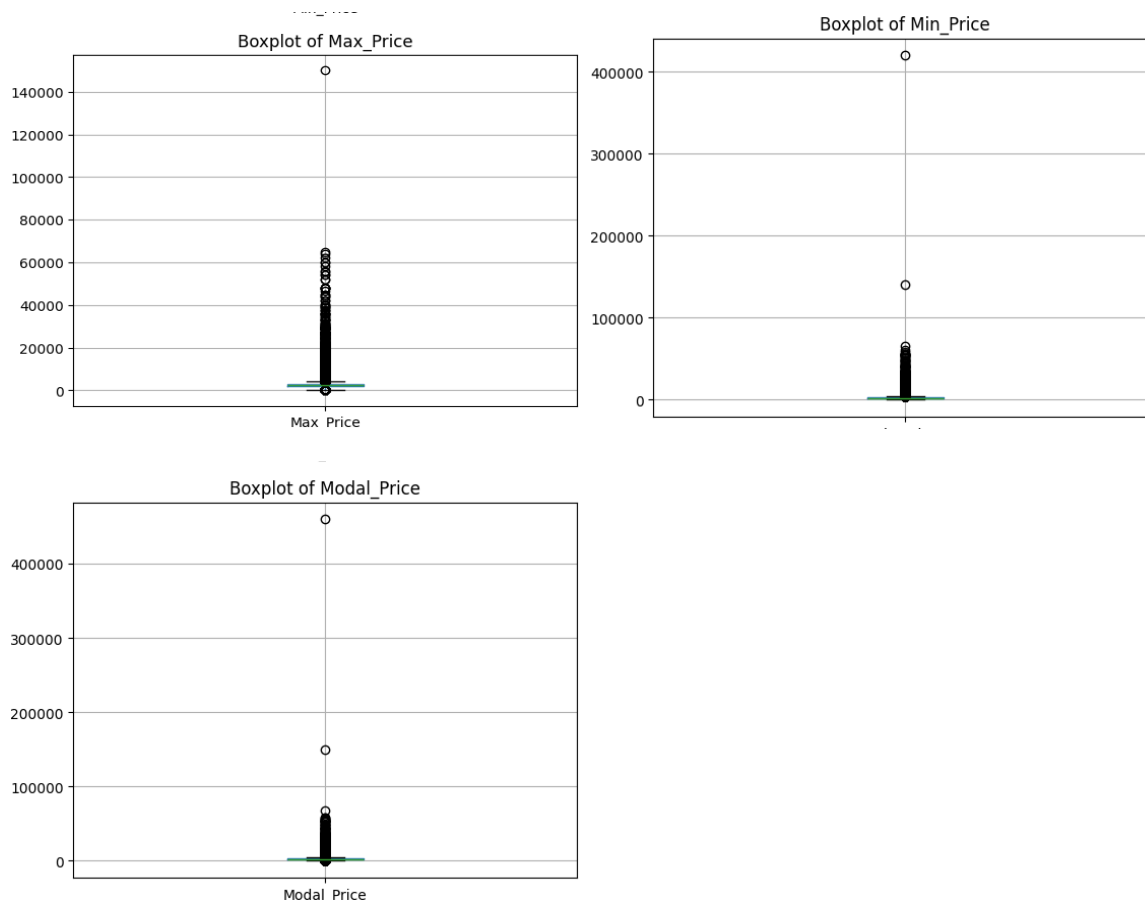
```
# using subset to identify the columns and keep first row and remove rest
df = df.drop_duplicates(subset=['State','District_Name','Market_Name','Commodity','Price_Date'],keep='first')
df
```

	State	District_Name	Market_Name	Commodity	Variety	Grade	Min_Price	Max_Price	Modal_Price	Price_Date
0	Maharashtra	nashik	Lasalgaon(Niphad)	Wheat	Maharashtra 2189	FAQ	2000.0	2300.0	2150.0	2023-06-06
1	Maharashtra	Unknown	Patan	Tomato	Other	FAQ	1000.0	1500.0	2150.0	2023-06-06
2	Uttar Pradesh	mainpuri	Unknown	Potato	Unknown	Unknown	800.0	820.0	2150.0	2023-06-06
4	Rajasthan	Unknown	Pratapgarh	Unknown	Other	FAQ	476.0	2300.0	617.0	2023-06-06
6	Haryana	ambala	Unknown	Unknown	Unknown	FAQ	500.0	800.0	700.0	2023-06-06
...	...	...	...	...	...	...	...	...	...	...
811109	Uttar Pradesh	balrampur	Unknown	Onion	Red	Unknown	2010.0	2050.0	2030.0	2024-01-10
811112	West Bengal	punuliya	Punulia	Onion	Other	FAQ	2300.0	2700.0	2500.0	2023-10-02
811120	Uttar Pradesh	Unknown	Chutmalpur	Potato	Potato	Unknown	1400.0	1600.0	1500.0	2023-09-03
811126	Tamil Nadu	nagapattinam	Nagapattinam(Uzhavar Sandhai)	Onion	Unknown	Local	4500.0	5000.0	5000.0	2024-11-01
811130	Jammu & Kashmir	Unknown	Unknown	Unknown	Nasik	FAQ	1400.0	1600.0	1500.0	2023-06-09

226498 rows x 10 columns

Numeric columns for check outliers: The code is used to find the outliers using boxplot.

```
# Check the Outliers using Box Plot #
price_cols = ['Min_Price', 'Max_Price', 'Modal_Price']
for col in price_cols:
    plt.figure()
    df.boxplot(column=col)
    plt.title(f'Boxplot of {col}')
    plt.show()
```



Min_price boxplot is concentrated in a narrow low range. However, a few extreme values increase the spread. These outliers may represent abnormal market conditions.

The Max_Price boxplot distribution is highly skewed with a large number of outliers. This extreme outlier may reflect unusual market conditions.

The Modal Price boxplot distribution is relatively stable and most value in a lower range, indicating consistent market pricing

Outliers Removed with Numerical Columns

The code automatically detects and tames outliers (extreme high or low values) in your price columns, so they don't skew your data analysis or machine learning models.

Finds the Middle 50%: It calculates Q1 (25th percentile) and Q3 (75th percentile) for a column.

Measures the Spread: It finds the **Interquartile Range (IQR)**, which is the distance between Q1 and Q3 ($IQR = Q3 - Q1$).

Sets the Boundaries: It establishes "normal" boundaries.

- Anything below $Q1 - 1.5 \times IQR$ is too low.
- Anything above $Q3 + 1.5 \times IQR$ is too high.

Clips the Extremes: The `clip(lower, upper)` function acts like a ceiling and a floor. Instead of deleting the outliers, it forces any value outside the boundaries to equal the exact maximum or minimum allowed value.

```
# Removing outliers using IQR. In this Method Replace extreme values with boundary limits #
price_cols = ['Min_Price', 'Max_Price', 'Modal_Price']
for col in price_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    # outliers = df[(df[col] < lower) | (df[col] > upper)] #
    # print(f"{col} outliers:", len(outliers)) #
    df[col] = df[col].clip(lower, upper)
```

Data Visualizations

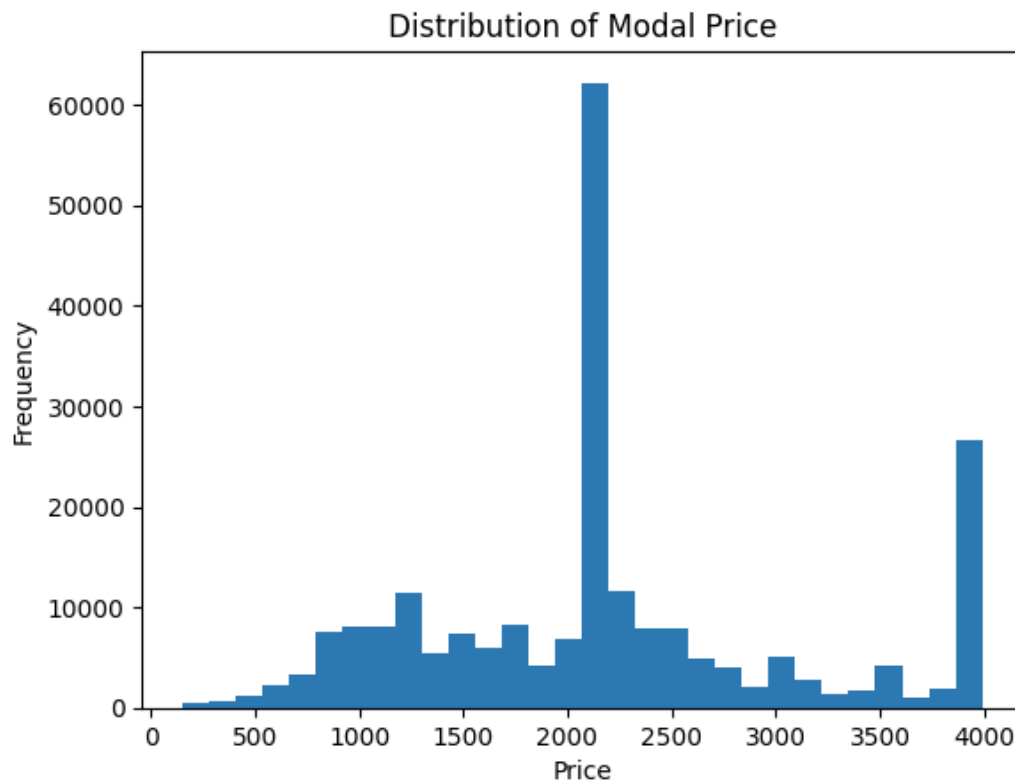
Univariate Analysis - Distribution of Single Variables

```
# Univariate Analysis Modal_Price #  
df['Modal_Price'].hist(bins=30)  
plt.title('Distribution of Modal Price')  
plt.xlabel('Price')  
plt.ylabel('Frequency')  
plt.grid()  
plt.show()
```

The histogram shows that most Modal Price values are concentrated in the lower price range and few only have higher price ranges.

Some occasional high price spikes, due to factors such as supply shortages, seasonal demand, or market variations.

The Conclusion of the chart is stable pricing, and occasional fluctuations



```
ax = sns.countplot(data=df,x='Commodity')
ax.bar_label(ax.containers[0])
plt.title('Top Commodity')
plt.show()
```

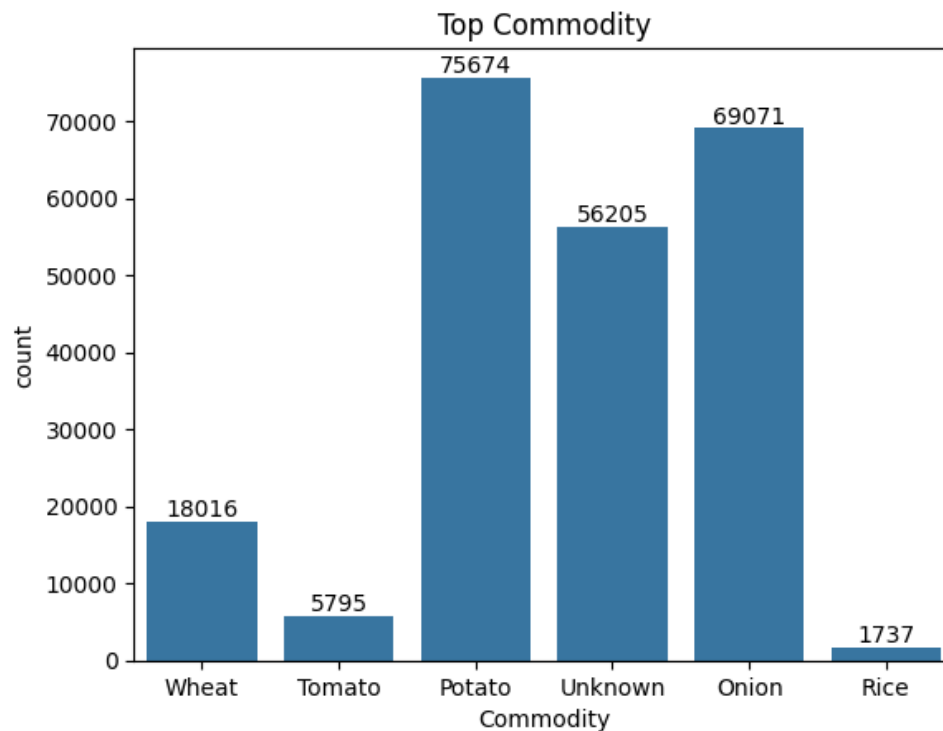
The chart shows the count of each commodity in the dataset.

Each bar represents a different commodity.

Bar labels display the exact count for each commodity

Commodities with taller bars are more frequently traded.

It provides a clear comparison of distribution across all commodities

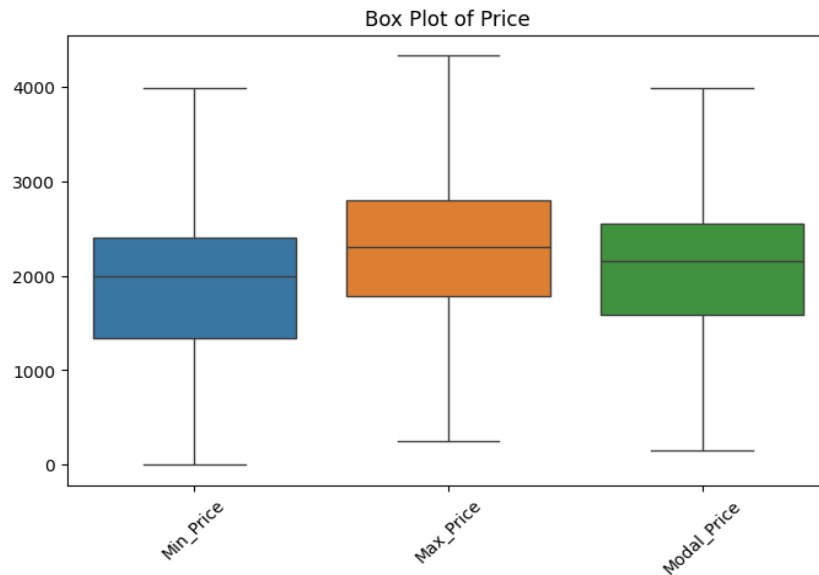


```
price_cols = ['Min_Price', 'Max_Price', 'Modal_Price']
plt.figure(figsize=(8,5))
sns.boxplot(data=df[price_cols])
plt.title('Box Plot of Price')
plt.xticks(rotation=45)
plt.show()
```

The box plot visualizes the distribution of price variables and helps identify the presence of outliers

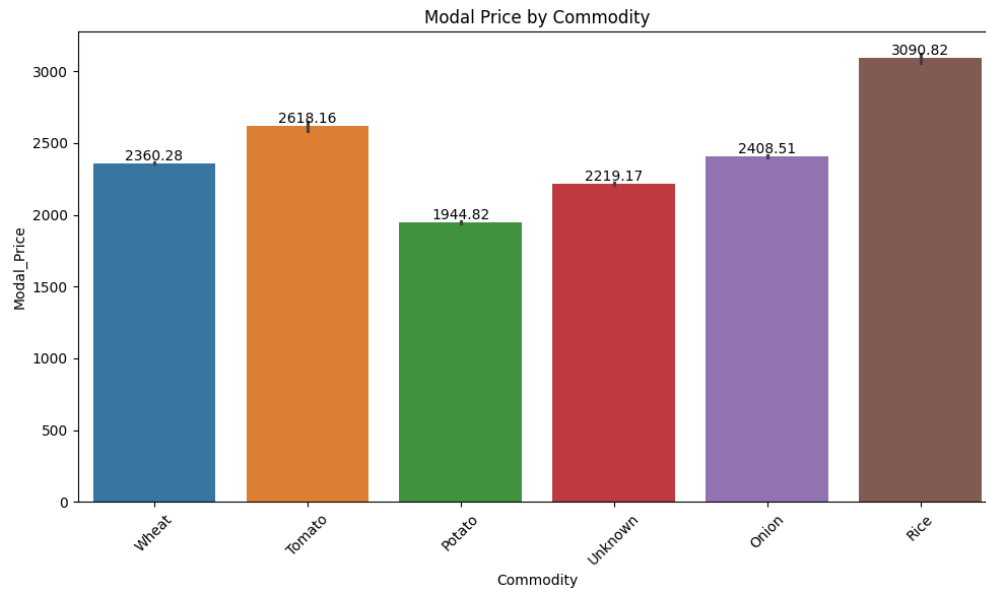
The box plot compares Min, Max, and Modal prices.

Presence of outliers suggests price fluctuations across market.



Bivariate Analysis - Relation between two variables

```
# Bar Plot is used to compare different commodity for actual market price i.e modal price #
plt.figure(figsize=(10,6))
ax = sns.barplot(data=df, x='Commodity', y='Modal_Price', hue='Commodity')
for container in ax.containers:
    ax.bar_label(container)
plt.title('Modal Price by Commodity')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

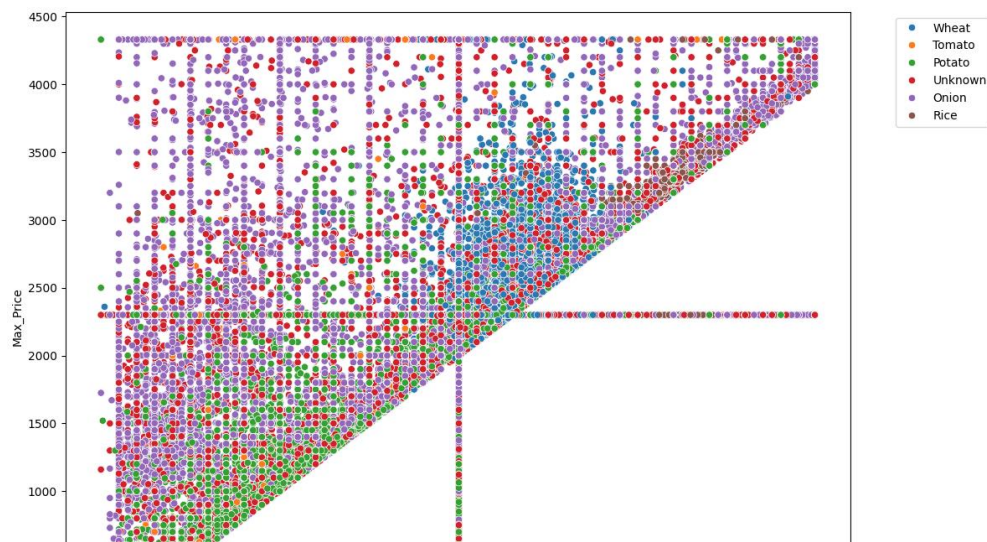


This Chart shows the relationship between Modal Price and Max Price

Most data are points positive trend, indicating that higher modal prices are associated with higher maximum prices.

Different colours represent different commodities, helping compare pricing patterns and identify price variations and outlier

```
# Scatter plot #
# compare relationship between the Min.and Max.price analyse price trends #
plt.figure(figsize=(12,8))
sns.scatterplot(data=df,x='Min_Price',y='Max_Price',hue='Commodity')
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()
```

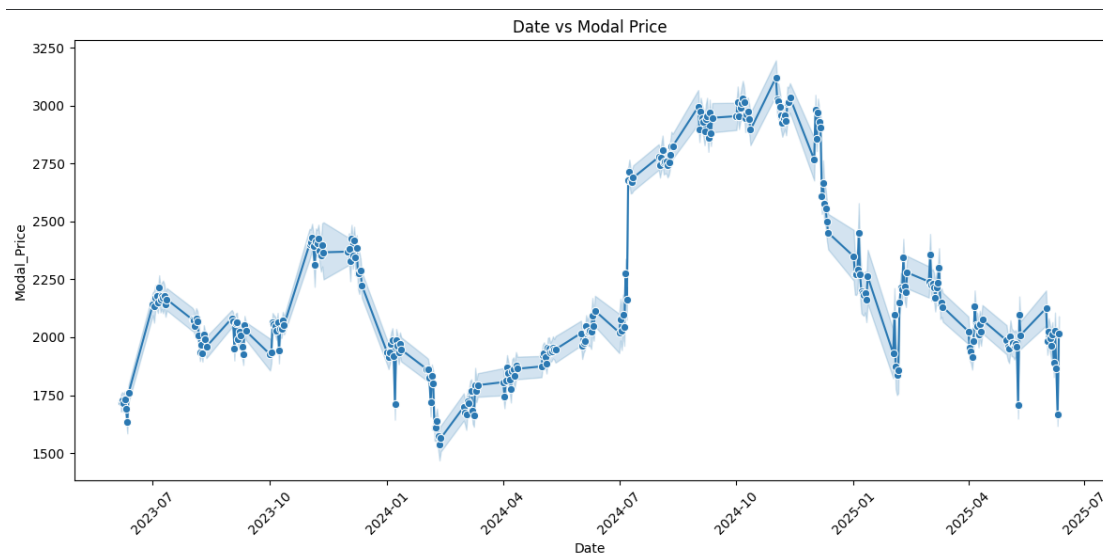


Minimum and Maximum prices are positively related

Different commodities show different pricing behaviours

Some commodities have stable prices, and few outliers suggest extreme price values for unusual market condition

```
# line chart defines how the prices changes over time #  
plt.figure(figsize=(12,6))  
sns.lineplot(data=df, x='Price_Date', y='Modal_Price', marker='o')  
plt.title('Date vs Modal Price')  
plt.xlabel('Date')  
plt.ylabel('Modal_Price')  
plt.xticks(rotation=45)  
plt.tight_layout()  
plt.show()
```



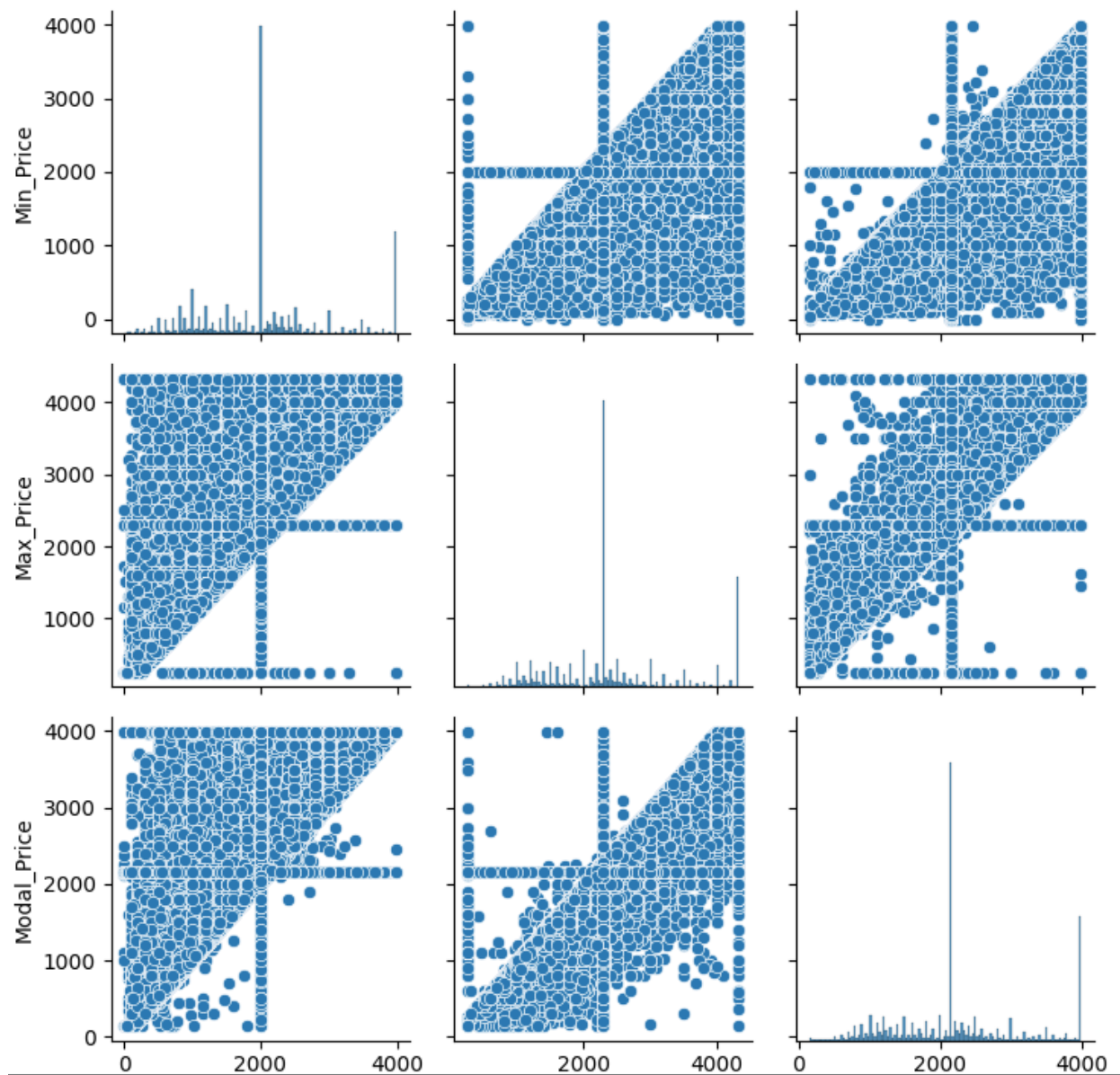
The chart shows how prices change over time. Stable lines indicate consistent pricing.

Upward and Downward trend indicates how prices are changed in the market for every day

Sudden spikes indicate low supply and High demand

Multivariate Analysis

```
# pair plot Relationship for multiple numerical column #  
sns.pairplot(df[['Min_Price', 'Max_Price', 'Modal_Price']], diag_kind='hist')  
plt.tight_layout()  
plt.show()
```

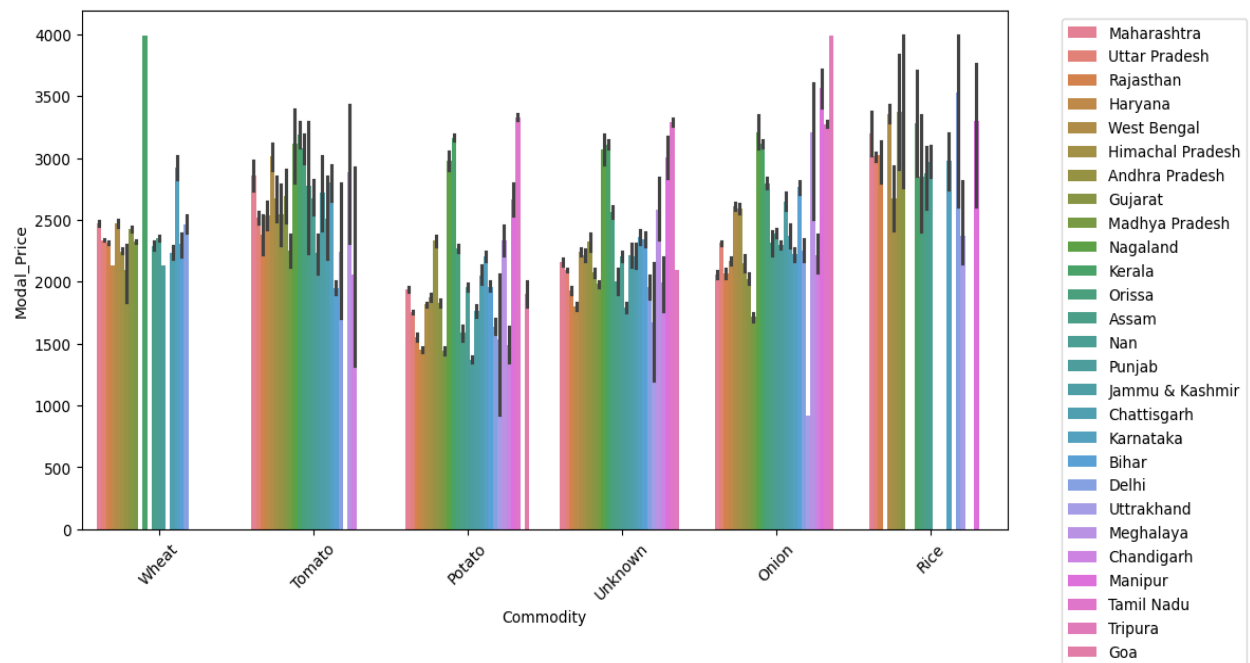


Displays all variable relationships together.

Diagonal for each prices indicates distribution of each variable.

This chart used for detecting outliers and clusters

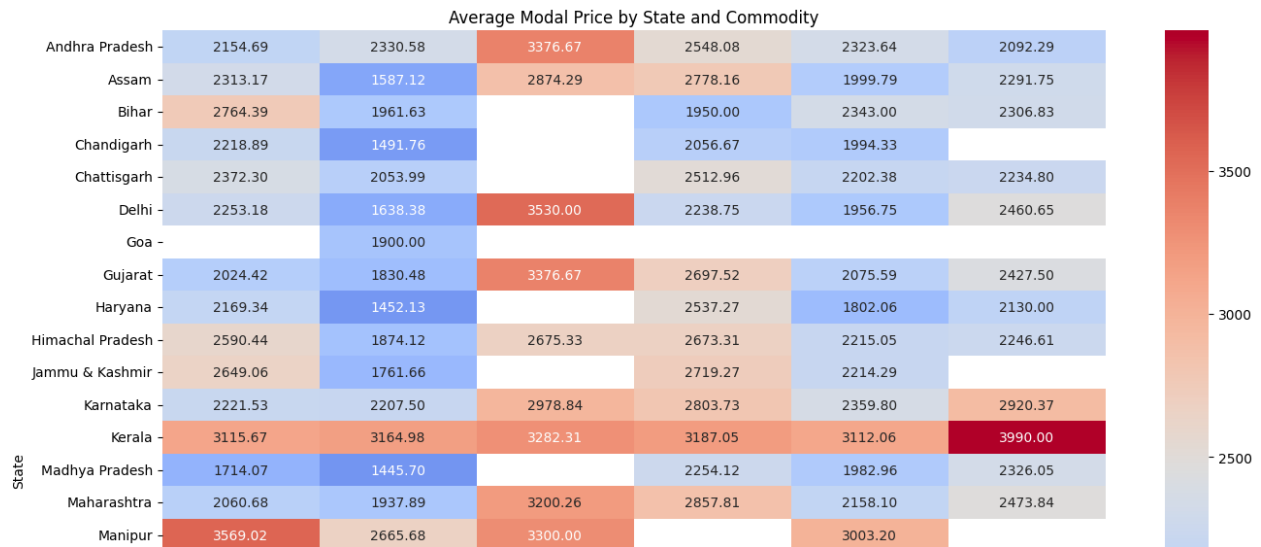
```
# Grouped Bar Plot is used to compare the prices of different commodities across various states #
plt.figure(figsize=(12,6))
sns.barplot(data=df,x='Commodity',y='Modal_Price',hue='State')
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



Compare the multiple variables together.

This chart is used to identify which state has highest prices.

It used to compare commodity wise price comparison

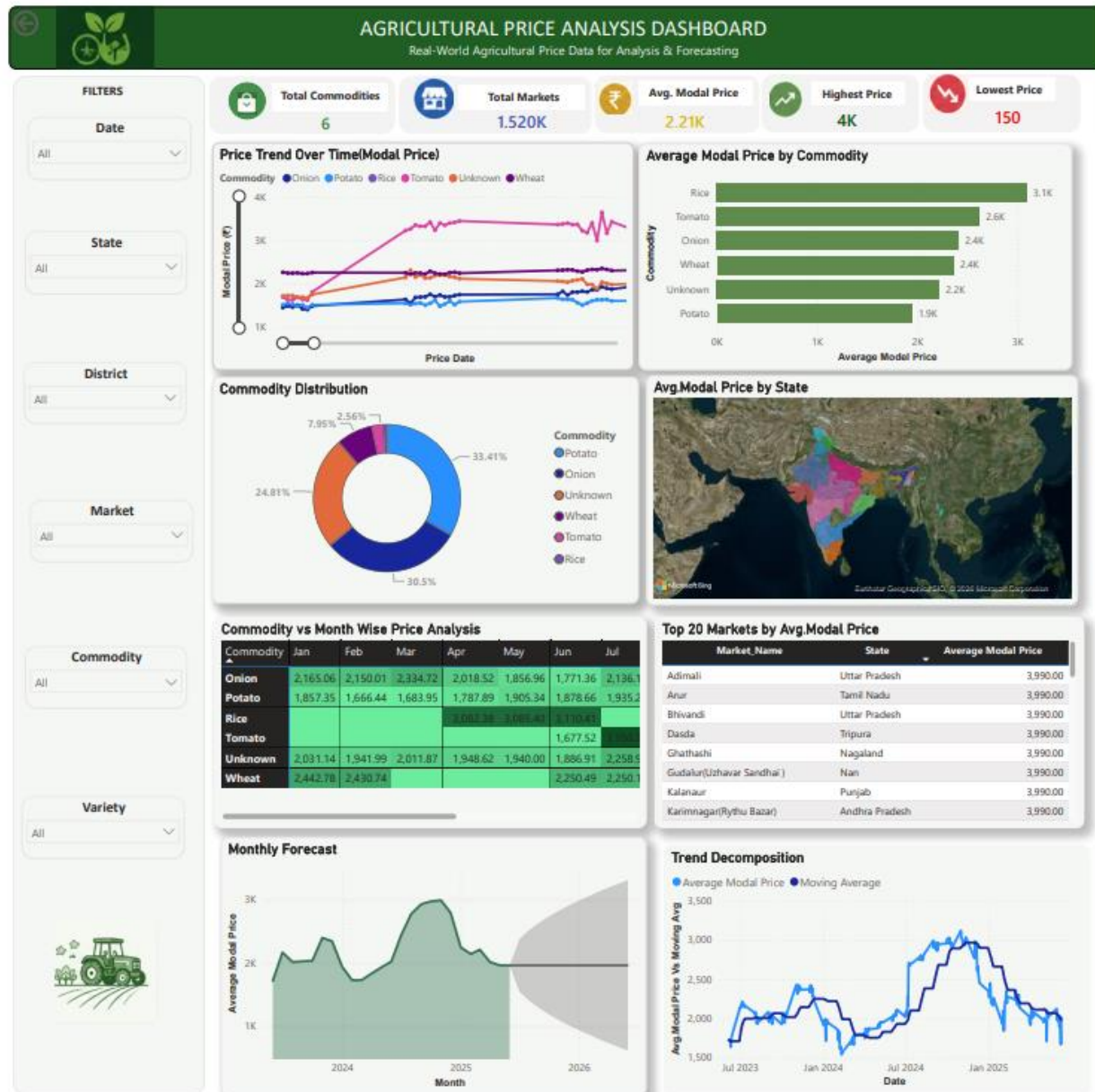


This heatmap represents the average modal prices of commodities across different states. colour scale blue indicates low prices and red for high prices

Kerala has high prices for almost all commodities.it indicates higher market demand and strong consumer market

In white blank cells Commodity unavailable in that state and no market records

Dashboard



Dashboard – Summary

This dashboard provides a complete overview of agricultural commodity prices across different states, markets, and time periods. It helps analyse pricing trends, commodity performance, market comparisons, and future forecasts.

Dashboard Objective

The dashboard is designed to:

- Monitor agricultural commodity prices
- Compare prices across states and markets
- Identify seasonal and monthly trends
- Analyse top-performing commodities
- Forecast future prices for decision-making

At the top of the Dashboard the KPI cards summarize important business insights.

Total Commodities – 6 Total number of agricultural commodities analyzed

Total Markets – 1.52K Total markets included in dataset

Avg. Modal Price – 2.21K Average selling price of all commodities

Highest Price – 4K Maximum recorded commodity price

Lowest Price – 150 Minimum recorded commodity prices

The slicers allow users to filter the entire dashboard dynamically and it can be used to analyze a specific commodity, Compare states and Study particular markets

- **Price Trend Over Time (Line Chart)** - This chart shows how modal prices change over time for different commodities
- **Average Modal Price by Commodity (Bar Chart)** - This Chart compares average prices of commodities
- **Commodity Distribution (Donut Chart)** - Shows percentage contribution of each commodity in the dataset.
- **Average Modal Price by State (Map Visual)** - Displays state-wise pricing on the India map
- **Commodity vs Month-wise Price Analysis (Heatmap Table)** - This matrix/heatmap shows monthly price variation for each commodity.
- **Top 20 Markets by Average Modal Price (Table)** - Ranks markets with highest average prices.
- **Monthly Forecast (Forecasting Visual)** - This chart predicts future commodity prices.
- **Trend Decomposition** - These visuals separate actual prices and moving average trends. It helps to identify long-term trends.

Key Insights:

Commodity Price Volatility - Some commodities show high fluctuations in prices across months and states. For example, Tomato prices often fluctuate due to perishability. Wheat and rice generally show more stable pricing patterns

State-wise Price Differences - The same commodity may have different prices in different states because of transport cost, climate conditions and local demand

Seasonal Price Patterns - Agricultural prices follow seasonal cycles for example tomato prices increase during off-season production periods and Rice prices stabilize after harvest seasons

Final Dashboard Conclusions

- Rice is the highest-priced commodity.
- Potato and Onion dominate market share.
- Prices show strong seasonal behavior.
- Certain states and markets consistently maintain higher prices.
- Forecasting suggests moderate price stability ahead.
- Trend decomposition reveals cyclical agricultural pricing patterns.